
Dissecting Hybrid Model Layers: Attention vs. Gated DeltaNet

Manas Joshi^{*1} John Cooper¹ Frederic Sala¹

Abstract

Transformers have been the center of focus for mechanistic interpretability, with many learned mechanisms identified. Recent work has studied non-Transformer architectures, such as Mamba. However, the mechanisms of Gated DeltaNet, specifically in pretrained hybrids, have not been thoroughly investigated. We analyze these layers when performing associative recall in pretrained models containing both Transformer and Gated DeltaNet layers. We find that each layer type learned to conduct different parts of the mechanism. Many Gated DeltaNet heads have a repeating gating pattern, indicating these heads treat each sentence separately. Notably, the DeltaNet layers were not storing the key-value pairs in their state. Through patching, Transformers are identified as the layers retrieving the correct value from the context. This division of labor in pretrained models provides insights into how best to structure hybrid models for the most interactions.¹

1. Introduction

Hybrid models, containing both Transformer and other non-Transformer layers, have grown in popularity due to their increased efficiency and performance across numerous tasks. These models have been tested on natural language benchmarks as well as on synthetic tasks, where in both situations, there are clear improvements over models containing only one layer type. For example, Nemotron-H (NVIDIA et al., 2025) stacks both Mamba2 and standard attention layers, resulting in a model with fewer parameters that has a higher accuracy on MMLU-Pro, as well as other benchmarks.

A newer choice for non-Transformer layers is Gated DeltaNet (GDN) (Yang et al., 2025). GDN has the sub-quadratic efficiency of other SSMs, while its parameterization is simple yet more expressive than other SSMs, such as

Mamba. These layers maintain a state where updates store a key-value pair for each new token in the sequence. The strength of long-term retention and the impact of the current key-value update are controlled by a pair of gating functions. This update mechanism makes these models amenable to easily storing relations, such as those in associative recall (Gupta et al., 2022). These layers have been integrated in a few frontier models, such as Qwen 3.5 and Kimi Linear.

Prior work focuses primarily on Transformer-only models (Rai et al., 2024), with a few newer works investigating specific SSM variants such as Mamba (Mohan et al., 2026). These analyses do not directly transfer to GDN, nor do they transfer to hybrids. Additionally, as is true for all attribution work for deep models, various components interact in complex, non-trivial ways, often confounding the true underlying mechanism within a model. **The internal workings of hybrid models with GDN layers in pretrained models are not well understood.**

In this work, we analyze one such pretrained family of hybrids, the Qwen 3.5 family, on associative recall. First, we investigate the component unique to Gated DeltaNet: its gates α and β . These gates profile the behavior of any given head, identifying at a high level what the head is doing. Next, we perform global interventions on these gates to test if GDN is storing key-value pairs to increase state update scale to prevent any possible noisy writes and reads. Lastly, we perform residual patching, verifying which layers are most responsible for retrieving the correct value from the context.

We observe that the gates across associative recall (Arora et al., 2026) prompts distinguish head-types from each other. The most common gating pattern observed interacts with each sentence separately, resetting its state after each sentence. The interventions increasing the write strength of GDN layers showed an improvement in performance by around 10%; however, this is far too small of an improvement if the GDN states were storing key-value pairs. Thus, despite the design of GDN layers to retain key-value pairs, we observe that in hybrid models with Transformer layers, *the Transformer layers perform the majority of retrieval in recall*. The attention components in these layers primarily bring the correct tokens in each recall pair into the response. Patching at each layer and each token in the context sepa-

¹Department of Computer Science, University of Wisconsin, Madison, USA. Correspondence to: Manas Joshi <mjoshi26@wisc.edu>.

¹Code can be found [here](#).

rately validates this claim. The layers where the important positions to patch shift the most are at the attention layers.

Beyond these specific models, this mechanism explains a phenomenon seen in prior work: hybrids with even a few transformer layers outperform models with no transformer layers (Waleffe et al., 2024). Beyond recent theoretical work establishing this (Cooper et al., 2026), this mechanism clearly shows how different layer types fulfill different roles in a model.

In summary, we make the following contributions:

- We find that in a pair of hybrid models, Gated DeltaNet layers do *not* maintain the key-value pairs when performing associative recall.
- We analyze the gating patterns of Gated DeltaNet, observing that many heads have a gather-and-aggregate structure similar to those in attention-only models (Bick et al., 2025).
- We identify a cause in learned hybrid models that including some Transformer layers improves performance.

2. Related Work

Mechanistic Interpretability. Much work in the mechanistic interpretability literature has focused on understanding Transformers and attention (Ameisen et al., 2025; Rai et al., 2024). Some more recent works have tackled understanding the mechanism learned within Mamba models (Mohan et al., 2026). This analysis focused heavily on Δ of Mamba, most closely related to the α gates in GDN. We expand on this, both analyzing the α and β gates of GDN and their behaviors, possibly determining the best locations to intervene on for steering.

One recent work looked into how pure Transformers and pure SSMs perform associative recall (Arora et al., 2026). They found that these two model architectures learned very different mechanisms based on the locations where residual patching resulted in the greatest change in performance. We take a similar approach, but on hybrid architectures with layers of each type. We also focus on pretrained hybrids rather than those trained specifically on associative recall.

Hybrids. A multitude of hybrid architectures have been investigated. Some recent ones have been trained at scales and performance comparable to other foundation models, such as Nemotron-H (NVIDIA et al., 2025) and Kimi Linear (Team et al., 2025). Empirically, the optimal layer composition of hybrids has been studied, showing that around 20% of layers being attention is optimal (Waleffe et al., 2024). However, the reasons behind this small number of attention layers greatly improving performance over pure SSM models is not well understood. This work takes a step in this

direction, identifying a task where learned hybrids rely on attention.

Recent work has established that distilling Transformers into hybrids is possible, especially for certain common components within pure Transformers (Bick et al., 2026). We find similar behaviors in learned hybrids, where GDN often performs gather and aggregate.

DeltaNet. DeltaNet (Yang et al., 2024) and its gated variant (Yang et al., 2025) are state-space models designed to implement a similar but more expressive version of linear attention. Some works have extended this formulation to iterate the delta rule at each token multiple times, which has provable expressivity guarantees (Siems et al., 2026). Some recent work has focused on the gates of GDN (Sun et al., 2026), adding complexity in the β gates. These show the importance and utility of gates within these models, making the gates a plausible component to study thoroughly.

3. Preliminaries

We begin with a brief background on the Gated DeltaNet mechanism and associative recall.

3.1. Gated DeltaNet

Gated DeltaNet is a state-space model (SSM); these are sequence-to-sequence maps that process tokens one at a time, maintaining a state S as its memory through the sequence. These models have the benefit of taking sub-quadratic time to process a sequence, unlike attention. Unlike other SSMs, GDN is an example of a more expressive, non-diagonal SSM (Siems et al., 2026), where the different dimensions of the state can interact with each other. This is in contrast to other SSMs such as Mamba, where each state dimension is independent of the others.

All state-space models can be described through an update rule and an output rule. The update takes the next token and updates the state, followed by the output rule using the updated state and the current token to produce an output.

For a sequence $(x_i)_{i=1}^L$ of tokens $x_i \in \mathbb{R}^d$, the state $S \in \mathbb{R}^{d \times d}$ for GDN is matrix-valued. The update and output rules are as follows:

$$\begin{aligned}\tilde{S}_i &= \alpha_i S_{i-1} \\ S_i &= \tilde{S}_i + \beta_i (v_i - \tilde{S}_i k_i) k_i^\top \\ o_i &= S_i q_i\end{aligned}$$

where $k_i = \frac{W_k x_i}{\|W_k x_i\|_2}$, $q_i = W_q x_i$, $v_i = W_v x_i$, and $\alpha_i \in [0, 1]$ and $\beta_i \in [0, 1]$ are gates. This should be interpreted as updating the current state with a rank-1 update in the direction of $k_i^\top$ that can be read by a query q_j at some later time j . The state S then becomes a smooth key-value

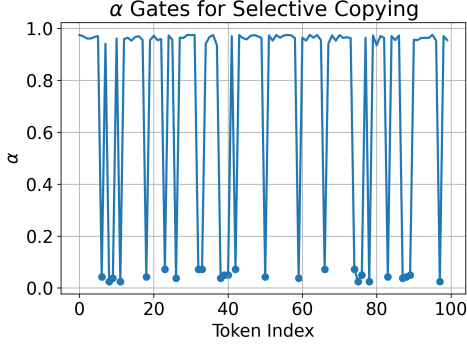


Figure 1. The α gates for a sample Selective Copying prompt. Dots represent the locations of number tokens. Number tokens reset the GDN state, while all other tokens maintain their state.

storage, where a test vector q_j will match the closest k_i 's and output their associated v_i .

At a high-level, the meaning of the gates are as follows:

- α_i : This controls how much to shrink every component of S_i . All currently stored values are shrunk equally. It is direction-agnostic.
- β_i : This controls how much of the old vector should be changed to the new value v_i . At a value of 1, the resulting state will satisfy $S_i k_i = v_i$. It is direction-aware.

3.2. Associative Recall

A well-known task required for strong performance on language tasks is associative recall (Arora et al., 2026). This task has been used to measure multiple models and their architectures against each other (Arora et al., 2024). Associative recall (AR) consists of a prompt containing pairs of tokens (k_i, v_i) as keys and values, along with a query q . The task is solved if the output is v_i where $k_i = q$. Transformers are known to perform this task with ease, even for two-layer models (Elhage et al., 2021).

We also consider another variant of associative recall where there are key duplicates in the context, making the task one of identifying the most recent key. This would be natural for Gated DeltaNet, but may be difficult for Transformers due to attention possibly attending to all relevant keys rather than the most recent one.

3.3. Synthetic Hybrid

For an initial toy experiment, we trained a small, two-layer GDN-Transformer hybrid on the task Selective Copying (Jelassi et al., 2024; Cooper et al., 2026). By looking at the gates, we can clearly see the mechanism learned through

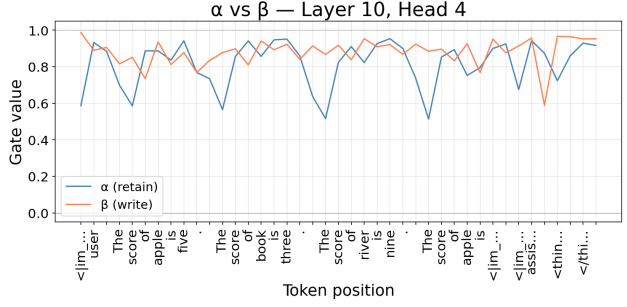


Figure 2. The gating profile for a select head from Qwen 3.5. This head depicts a commonly observed profile, with similar behavior for each new sentence.

the α gate, seen in Figure 1. We observe that at the positions of the number tokens, the model resets its state. At other positions, the model keeps α near 1, losing no stored information. Thus, a simple analysis of the gates of GDN heads can provide insight into their mechanism.

4. Results

Beyond models trained on tasks with easily interpretable gates, we analyzed larger models pretrained and instruction-tuned on general language datasets. We use the Qwen3.5 instruction-tuned family of models. Unless otherwise specified, results are shown for the 0.8B parameter model. Results for other models can be found in Appendix C. We look at the gates in GDN layers and residual patching to distinguish the behavior of each layer type.

4.1. Gating Behavior

The gates α and β govern very distinct behavior in the GDN formulation. Thus, these two gates provide a lens into the inner workings of GDN layers, especially for interpretable tasks such as associative recall. We demonstrate the utility of these gate profiles by passing an associative recall prompt into Qwen 3.5 and capturing the gates at each token for each head separately.

The gating profile of a common head can be seen in Figure 2. These heads repeat similar gating patterns for each new sentence they process, typically with a decreased α at sentence boundaries. The behavior is similar to the Gather and Aggregate mechanism (Bick et al., 2025), where GDN is summarizing sentences for later layers to process more readily. Other profiles observed are found in Appendix B.

4.2. Ablations

To corroborate the previous finding of Gather and Aggregate GDN heads, we performed ablations on every head within Qwen 3.5 0.8B individually. For each ablation, the accuracy on 10 different associative recall prompts was measured.

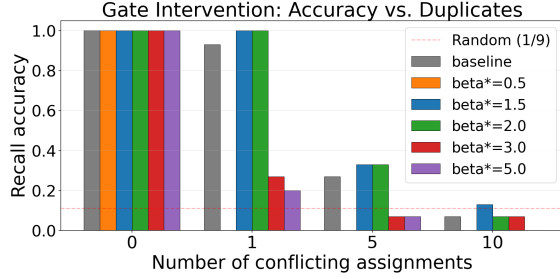


Figure 3. Different gate interventions and their effects on recall. Input-agnostic effects are seen in the right of each cluster. The larger multiplication of β improves recall more.

Results can be observed in Figure 10. The relevant entry is for layer 10, head 4, the same as seen in Figure 2. When ablating this head, the model’s ability to perform associative recall is entirely broken, failing on every prompt. It is likely that this head is responsible for aggregating information across sentences, while attention layers are responsible for retrieval. A detailed connection between these two layers, as well as an automated method to connect these two layers as a circuit, is part of future work.

4.3. Gate Interventions

We also studied a few gate interventions to test their effects on the ability of GDN hybrids to perform recall. The gate responsible for writing new information is β . A simple intervention that may improve recall is increasing β by some factor, increasing the write strength. Specifically, we test these effects on associative recall with duplicate keys. If a Gated DeltaNet layer is storing the associative recall pairs, completely overwriting the original assignment (i.e., setting $\beta = 1$) should push performance up significantly.

The results are found in Figure 3. With no duplicate keys, all interventions work with perfect performance. However, as the number of conflicting assignments grows, base model performance deteriorates. Scaling β by some factor shows some improvements over the base model. However, these improvements are quite weak.

Thus, these results indicate that *GDN is not primarily responsible for recall*, at least not value retrieval. If a Gated DeltaNet layer was storing the key-value pairs, this intervention would be stronger. Interventions that set $\beta = 0$ on all but the last, correct assignment can be found in Appendix C. These corroborate other experiments corroborate this result, including proof that GDN is performing an important part of recall, but not value retrieval.

4.4. Patching

To further validate that attention is responsible for retrieval in recall tasks, we take a recall prompt and a corruption

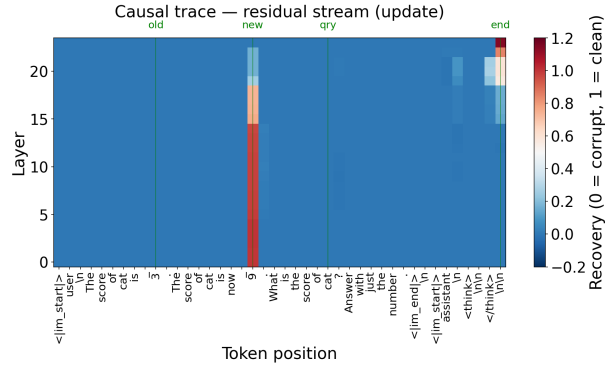


Figure 4. Effects on patching individual layers at specific token positions within Qwen 3.5 0.8B on a corrupted prompt. The impactful tokens shift most strongly at layer 15 and 19, both attention layers.

where the target value is changed. We then input the corrupted prompt to the model, but patch the residual of the correct prompt at different layers and different token positions.

From this, we should observe the following. First, at early layers, the residual at the value’s location will be the most critical as the model has not moved that token’s information to the end of the context. Similarly, at late layers, patching the residual of the last token (which will be decoded) will impact the result strongly. Beyond these two predictions, any structures will indicate where in the context the model is pulling information from to perform this recall task.

The results are seen in Figure 4. Both of the expected behaviors are observed. Additionally, we observe that the patching influence shifts from value token to final token at 3 layers most strongly: layers 15, 19, and 23. *These are all attention layers*. If GDN was responsible for retrieving the correct value to the end of the context, we would see the impact shift to the final token at a non-Transformer layer.

Also, observe that some tokens in the thinking phase of the context influence the output. These are suspected to be artifacts of instruction tuning as they occur around the additional tokens introduced from this tuning.

5. Conclusion

In the pretrained DeltaNet hybrid Qwen 3.5, we observed that attention and GDN learn to perform distinct roles. The primary mechanism for retrieval is still attention, while GDN plays a non-negligible role in performing recall. The gates of GDN provide a lens to view the inner workings of these layers, depicting a standard Gather and Aggregate behavior for recall. For future work, other tasks can be analyzed in the same way, possibly leading to a characterization of many of the learned heads as sentence summarizers.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Ameisen, E., Lindsey, J., Pearce, A., Gurnee, W., Turner, N. L., Chen, B., Citro, C., Abrahams, D., Carter, S., Hosmer, B., Marcus, J., Sklar, M., Templeton, A., Bricken, T., McDougall, C., Cunningham, H., Henighan, T., Jermyn, A., Jones, A., Persic, A., Qi, Z., Ben Thompson, T., Zimmerman, S., Rivoire, K., Conerly, T., Olah, C., and Batson, J. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- Arora, A., Rathi, N., Selvam, N. R., Csordás, R., Jurafsky, D., and Potts, C. Mechanistic evaluation of transformers and state space models, 2026. URL <https://arxiv.org/abs/2505.15105>.
- Arora, S., Eyuboglu, S., Timalsina, A., Johnson, I., Poli, M., Zou, J., Rudra, A., and Re, C. Zoology: Measuring and improving recall in efficient language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=LY3ukUANKo>.
- Bick, A., Xing, E. P., and Gu, A. Understanding the skill gap in recurrent models: The role of the gather-and-aggregate mechanism. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=hWYisuBbp7>.
- Bick, A., Xing, E. P., and Gu, A. Retrieval-aware distillation for transformer-ssm hybrids, 2026. URL <https://arxiv.org/abs/2602.11374>.
- Cooper, J., Diakonikolas, I., Ma, M., and Sala, F. Expressivity-efficiency tradeoffs for hybrid sequence models, 2026. URL <https://arxiv.org/abs/2603.08859>.
- Elhage, N., Nanda, N., Olsson, C., Henighan, T., Joseph, N., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., DasSarma, N., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S., and Olah, C. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. <https://transformer-circuits.pub/2021/framework/index.html>.
- Gupta, A. et al. Transformers are provably efficient for sequence modeling. *arXiv preprint arXiv:2208.04933*, 2022.
- Jelassi, S., Brandfonbrener, D., Kakade, S. M., and Malach, E. Repeat after me: transformers are better than state space models at copying. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- Mohan, V. S., Gupta, K., Das, A., and Singh, C. Interpreting and steering state-space models via activation subspace bottlenecks, 2026. URL <https://arxiv.org/abs/2602.22719>.
- NVIDIA, :, Blakeman, A., Basant, A., Khattar, A., Renduchintala, A., Bercovich, A., Ficek, A., Bjorlin, A., Taghibakhshi, A., Deshmukh, A. S., Mahabaleshwarkar, A. S., Tao, A., Shors, A., Aithal, A., Poojary, A., Dattagupta, A., Buddharaju, B., Chen, B., Ginsburg, B., Wang, B., Norick, B., Butterfield, B., Catanzaro, B., del Mundo, C., Dong, C., Harvey, C., Parisien, C., Su, D., Korzekwa, D., Yin, D., Gitman, D., Mosallanezhad, D., Narayanan, D., Fridman, D., Rekesh, D., Ma, D., Pykhtar, D., Ahn, D., Riach, D., Stosic, D., Long, E., Segal, E., Evans, E., Chung, E., Galinkin, E., Bakhturina, E., Dobrowolska, E., Jia, F., Liu, F., Prasad, G., Shen, G., Liu, G., Chen, G., Qian, H., Ngo, H., Liu, H., Li, H., Gitman, I., Karmanov, I., Moshkov, I., Golan, I., Kautz, J., Scowcroft, J. P., Casper, J., Seppanen, J., Lu, J., Sewall, J., Zeng, J., You, J., Zhang, J., Zhang, J., Huang, J., Xue, J., Huang, J., Conway, J., Kamalu, J., Barker, J., Cohen, J., Jennings, J., Parmar, J., Sapra, K., Briski, K., Chumachenko, K., Luna, K., Santhanam, K., Kong, K., Sivamani, K., Pawelec, K., Anik, K., Li, K., McAfee, L., Derczynski, L., Pavao, L., Vega, L., Voegtli, L., Bala, M., de Melo, M. R., Sreedhar, M. N., Chochowski, M., Kliegl, M., Stepniewska-Dziubinska, M., Le, M., Novikov, M., Samadi, M., Andersch, M., Evans, M., Martinez, M., Chrzanowski, M., Ranzinger, M., Blaz, M., Smelyanskiy, M., Fawzy, M., Shoeby, M., Patwary, M., Lee, N., Tajbakhsh, N., Xu, N., Rybakov, O., Kuchaiev, O., Delalleau, O., Nitski, O., Chadha, P., Shamis, P., Micikevicius, P., Molchanov, P., Dykas, P., Fischer, P., Aquilanti, P.-Y., Bialecki, P., Varshney, P., Gundecka, P., Tredak, P., Karimi, R., Kandou, R., El-Yaniv, R., Joshi, R., Waleffe, R., Zhang, R., Kavanaugh, S., Jain, S., Krizan, S., Lym, S., Satheesh, S., Muralidharan, S., Narendhiran, S., Anandaram, S., Bak, S., Kashirsky, S., Han, S., Acharya, S., Ghosh, S., Sreenivas, S. T., Clay, S., Thomas, S., Prabhume, S., Pachori, S., Toshniwal, S., Prayaga, S., Jain, S., Das, S., Kierat, S., Majumdar, S., Han, S., Singhal, S., Niverty, S., Alborghetti, S., Panguluri, S., Bhendigeri, S., Akter, S. N., Migacz, S., Shiri, T., Kong, T., Roman, T., Ronen, T., Saar, T., Konuk, T., Rintamaki,

- T., Poon, T., De, U., Noroozi, V., Singh, V., Korthikanti, V., Kurin, V., Ahmad, W. U., Du, W., Ping, W., Dai, W., Byeon, W., Ren, X., Xu, Y., Choi, Y., Zhang, Y., Lin, Y., Suhara, Y., Yu, Z., Li, Z., Li, Z., Zhu, Z., Yang, Z., and Chen, Z. Nemotron-h: A family of accurate and efficient hybrid mamba-transformer models, 2025. URL <https://arxiv.org/abs/2504.03624>.
- Rai, D., Zhou, Y., Feng, S., Saparov, A., and Yao, Z. A practical review of mechanistic interpretability for transformer-based language models. *arXiv preprint arXiv:2407.02646*, 2024.
- Siems, J., Carstensen, T., Zela, A., Hutter, F., Pontil, M., and Grazi, R. Deltaproduct: Improving state-tracking in linear RNNs via householder products. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=SoRiaijTGr>.
- Sun, P., Hu, Y., Tan, J., Wang, X., Zhang, J., Lu, Y., Sun, Y., Xie, Y., and Cai, X. Fg²-gdn: Enhancing long-context gated delta networks with doubly fine-grained control, 2026. URL <https://arxiv.org/abs/2604.19021>.
- Team, K., Zhang, Y., Lin, Z., Yao, X., Hu, J., Meng, F., Liu, C., Men, X., Yang, S., Li, Z., Li, W., Lu, E., Liu, W., Chen, Y., Xu, W., Yu, L., Wang, Y., Fan, Y., Zhong, L., Yuan, E., Zhang, D., Zhang, Y., Liu, T. Y., Wang, H., Fang, S., He, W., Liu, S., Li, Y., Su, J., Qiu, J., Pang, B., Yan, J., Jiang, Z., Huang, W., Yin, B., You, J., Wei, C., Wang, Z., Hong, C., Chen, Y., Chen, G., Wang, Y., Zheng, H., Wang, F., Liu, Y., Dong, M., Zhang, Z., Pan, S., Wu, W., Wu, Y., Guan, L., Tao, J., Fu, G., Xu, X., Wang, Y., Lai, G., Wu, Y., Zhou, X., Yang, Z., and Du, Y. Kimi linear: An expressive, efficient attention architecture, 2025. URL <https://arxiv.org/abs/2510.26692>.
- Waleffe, R., Byeon, W., Riach, D., Norick, B., Korthikanti, V., Dao, T., Gu, A., Hatamizadeh, A., Singh, S., Narayanan, D., Kulshreshtha, G., Singh, V., Casper, J., Kautz, J., Shoeybi, M., and Catanzaro, B. An empirical study of mamba-based language models. In *ArXiv Preprint*, June 2024.
- Yang, S., Wang, B., Zhang, Y., Shen, Y., and Kim, Y. Parallelizing linear transformers with the delta rule over sequence length. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=y8Rm4VNRPH>.
- Yang, S., Kautz, J., and Hatamizadeh, A. Gated delta networks: Improving mamba2 with delta rule. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=r8H7xhYPwz>.

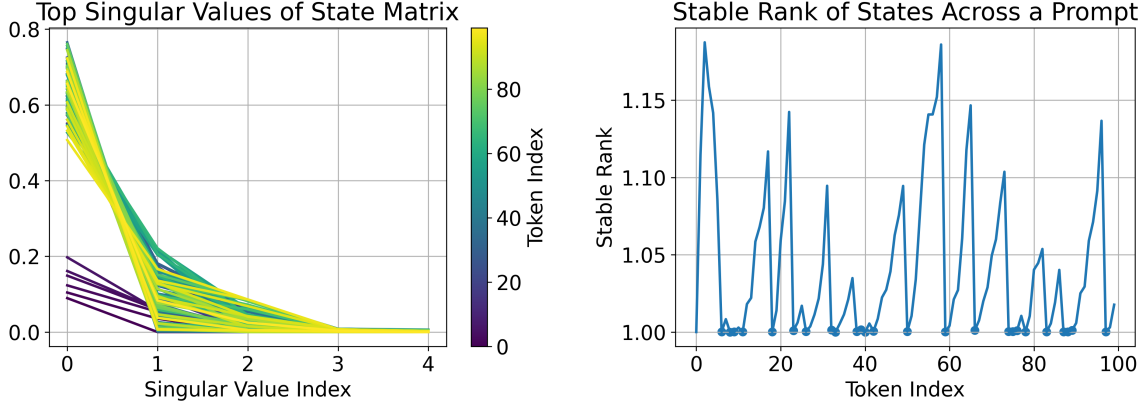


Figure 5. The spectra and related stable rank of the different states for a single prompt. Blue dots indicate positions of number tokens. The third singular value remains consistently low, and the stable rank is consistently reset to 1 at each new number token.

A. Experiment Details

We first share a few additional results for the synthetic Selective Copying experiment above. Code for reproducibility can be found at <https://anonymous.4open.science/r/deltanet-workshop-2020/README.md>.

A.1. Selective Copying

For these experiments, we trained a small, 2-layer hybrid on Selective Copying, as in (Cooper et al., 2026). However, a GDN head was used instead of Mamba, as was primarily investigated in that work. In summary, Selective Copying contains prompts of both number tokens and other tokens. The task is to take the last number token n and output the token n from the end of the context. This task has been shown to be provably easier for SSM-Transformer hybrids than either model on their own.

For a Selective Copying prompt, we captured the state at each token. We then analyzed the singular values and the stable rank at each token position. The results can be seen in Figure 1, and additional results can be seen in Figure 5. In the latter, we observe that the stable rank of these different states sharply decreases to 1 at each number token (represented by blue dots). This follows the observed α gate pattern, where the previous state is forgotten at each number token, leaving only a rank 1 component in the state.

Additionally, the top singular values show that the state never stores information of more than rank 3. This suggests a mechanism where the GDN layer is storing the previous number token as well as some information about the current token. However, it is not storing information about the previous n tokens. These imply that this GDN-Transformer hybrid is learning the mechanism where GDN is memorizing the last number while the Transformer is selecting the correct token from the context.

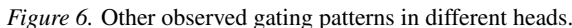
A.2. Associative Recall

Our associative recall prompts are constructed from natural language. Sentences of the form “The score of <object> is <number>” are combined in series. In certain situations, we allow for there to be multiple objects within the prompt to test if the most recent value is retrieved for a specific key. Figures 2 and 4 show the sampled prompt on the x-axis.

When measuring performance in Figure 3, the prompt ends with the instruction “Answer in one word.” We accept a response as correct if, after lowercasing, stripping whitespace, and removing a trailing period, it equals the target score either as a digit (e.g., “5”) or as its English word form (e.g., “five”). Anything longer (e.g., “The answer is five”) is marked incorrect.

B. Gate Types

Within Qwen 3.5, heads often followed consistent patterns for each new sentence in the context. Figure 2 shows a very standard behavior where the ends of sentences are treated substantially differently from their middles.



C. Additional Experiments

C.1. Qwen 3.5 2B Instruction Tuned

Similarly to Figure 3, we did the same experiment for the 2B model with results in Figure 8. As with the 0.8B model, small improvements in performance can be seen when scaling all of the β gates in the model.

The head-wise residual patching results for the base model can be seen in Figure 9. Unlike the instruction-tuned model, there are no thinking tokens, and layer 15 does not show the same behavior as the instruction-tuned model. However, the major shift in influence still occurs at layer 19, an attention layer.

We additionally conducted head-wise ablations for 10 prompts and measured the number of incorrect responses. The heads were sorted by performance. The attention layers have only 8 heads, while GDN has 16 heads per layer.

8

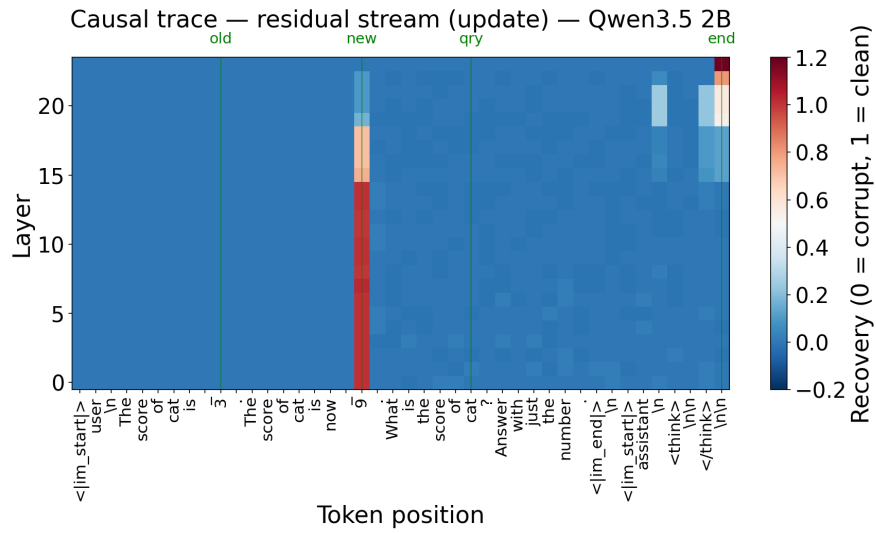


Figure 7. Residual patching results for Qwen 3.5 2B on a corrupted prompt.

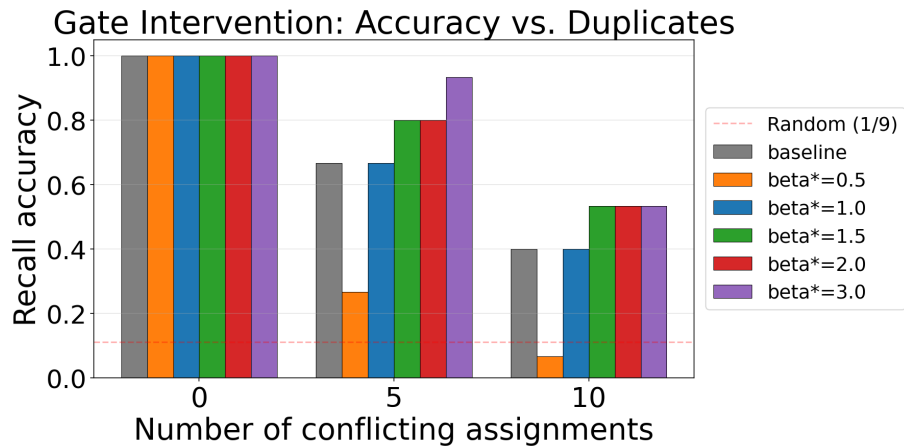


Figure 8. Gate intervention results for Qwen 3.5 2B.

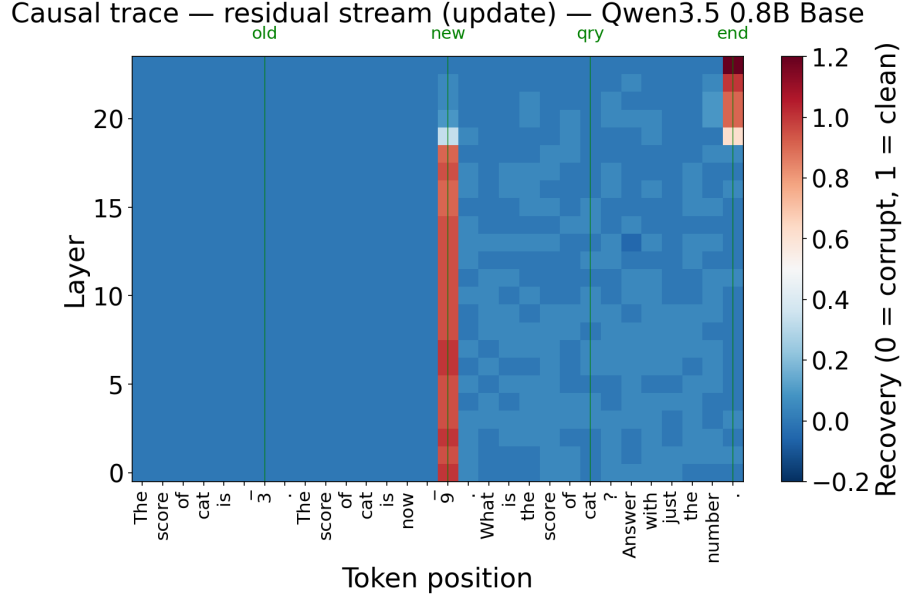


Figure 9. Residual patching results for Qwen 3.5 0.8B, base model, on a corrupted prompt.

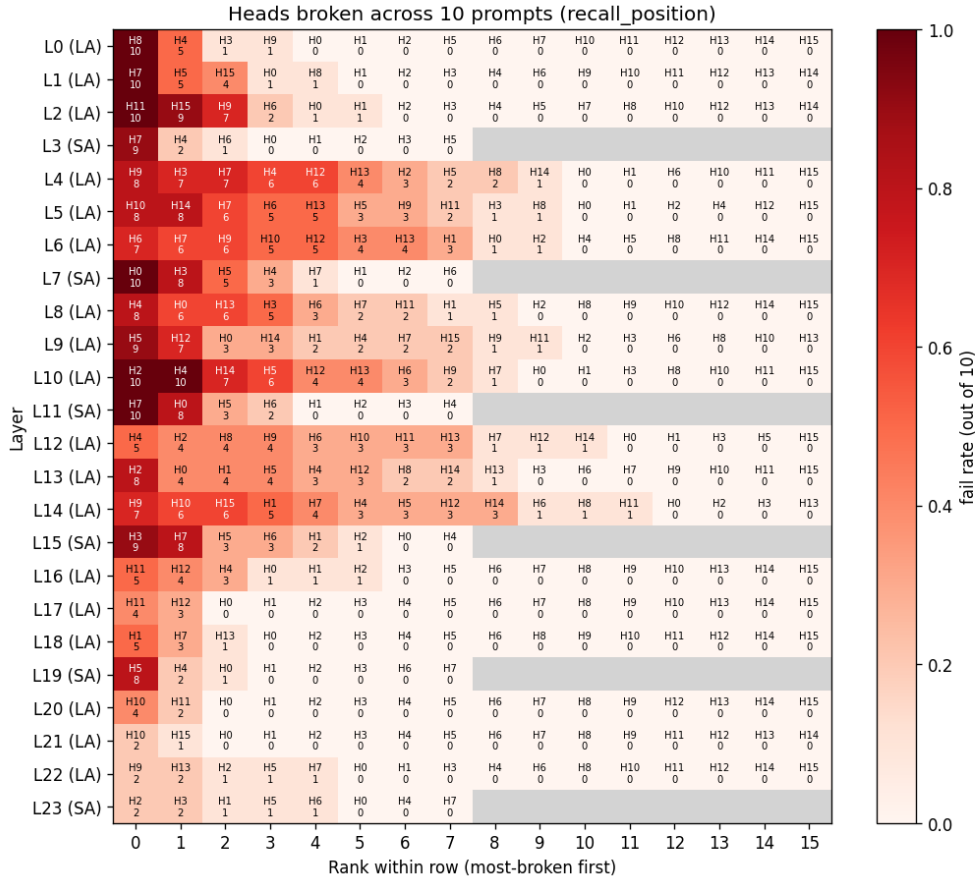


Figure 10. Ablations of each head in Qwen 3.5 0.8B. Shown are the count of incorrect responses after ablating a specific head, out of 10 associative recall prompts.

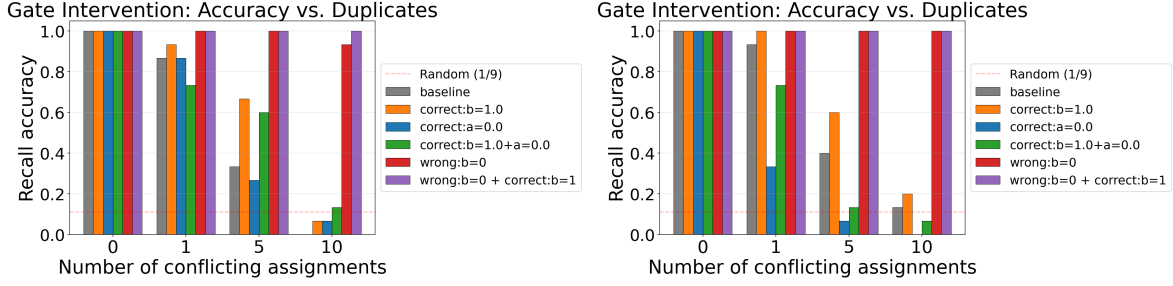


Figure 11. Interventions on specific locations in the prompt. The left plot is for the base 0.8B model, and the right is for the instruction-tuned 0.8B model.

C.4. Input-aware Gating

There are two types of gate interventions we experiment with: input-aware and input-agnostic. For input-aware interventions, the β values at the locations of the correct and/or incorrect assignments of a given key are modified. For input-agnostic interventions, β is affected at all positions in the context equally. We expect input-aware interventions to be most impactful as they affect different parts of the prompt separately, while input-agnostic interventions should be weaker.

The results for this are seen in Figure 11. Setting $\beta = 0$ on conflicting key-value pairs improves performance significantly. However, this would affect both the mechanism by which GDN maintains the key-value pairs as well as the mechanism where the residual after a GDN layer is read by a later Transformer layer. The experiment setting $\alpha = 0$ on the correct location shows that the former is incorrect. If the key-value is stored in the GDN states, then setting $\alpha = 0$ resets the state, removing all previous noise. However, this does not improve performance.